

Gioush 大队 2026 夏季常规赛

夏季训练赛

GSOI 2026

第十二试

时间：2026 年 8 月 3 日 14:00 ~ 17:30

题目名称	抢修计划	风眼定位	环锁解除	风暴止息
题目类型	传统型	交互型	传统型	传统型
目录	repair	locate	circle	calm
可执行文件名	repair	locate	circle	calm
输入文件名	repair.in	无	circle.in	calm.in
输出文件名	repair.out	无	circle.out	calm.out
每个测试点时限	1.0 秒	8.0 秒	1.5 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	20	3	25	25
测试点是否等分	是	否	是	是

提交源程序文件名

对于 C++ 语言	repair.cpp	locate.cpp	circle.cpp	calm.cpp
-----------	------------	------------	------------	----------

编译选项

对于 C++ 语言	-O2 -std=c++14
-----------	----------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 main() 的返回值类型必须是 int，程序正常结束时的返回值必须是 0。
3. 因违反以上两点而出现的错误或问题，申诉时一律不予受理。
4. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
5. 选手提交的程序源文件必须不大于 100KB。
6. 程序可使用的栈空间内存限制与题目的内存限制一致。

抢修计划 (repair)

【题目背景】

主营地防线演习结束很久以后，阳虹海上出现了一场持续扩大的异常风暴，沿岸观测设施开始成片失去响应。Rainbow_DDD 赶到海岸时，剩余时间已经不足以完成全部工作。为了让后续调查能够继续，Rainbow_DDD 必须先恢复沿岸设施的基本联系，再决定如何利用余下时间加固最重要的区域。

【题目描述】

沿岸共有 n 个抢修区域，依次编号为 $1 \sim n$ 。这些区域沿维护通道依次延伸，越靠后的区域越接近风暴中心。

最初只有区域 1 可以进入。对于 $i > 1$ ，只有编号为 $1 \sim i - 1$ 的区域都至少完成过一次抢修后，通往区域 i 的维护通道才会恢复，区域 i 也会立刻变为可以进入的状态。

每次行动中，Rainbow_DDD 可以选择一个当前能够进入的区域进行抢修，同一个区域可以被选择多次。

第一次抢修区域 i 时，会恢复该区域的基础设施，并获得 a_i 点保护价值。此后再次选择区域 i 时，只会继续加固已经恢复的设施，每次获得 b_i 点保护价值。

一个区域完成第一次抢修后将始终保持恢复。Rainbow_DDD 不需要在新区域开放后立刻前往该区域，可以继续抢修任意一个已经开放的区域，也可以随时结束行动。

风暴即将登陆，Rainbow_DDD 至多能够完成 k 次行动。请你求出能够获得的最大保护价值。

【输入格式】

从文件 `repair.in` 中读入数据。

本题包含多组测试数据。

输入的第一行包含两个非负整数 c, T ，分别表示测试点编号与测试数据的组数。 $c = 0$ 表示该测试点为样例。

对于每组测试数据：

第一行包含两个正整数 n, k ，分别表示抢修区域数量与最多能够完成的行动次数。

第二行包含 n 个正整数 $a_1, a_2, \dots, a_n$ 。

第三行包含 n 个正整数 $b_1, b_2, \dots, b_n$ 。

【输出格式】

输出到文件 `repair.out` 中。

对于每组测试数据输出一行一个整数，表示能够获得的最大保护价值。

【样例 1 输入】

```
1 0 3
2 4 7
3 4 3 1 2
4 1 1 1 1
5 3 2
6 1 2 5
7 3 1 8
8 5 5
9 3 2 4 1 4
10 2 3 1 4 7
```

【样例 1 输出】

```
1 13
2 4
3 15
```

【说明/提示】**【样例 1 解释】**

第一组测试数据中，可以依次抢修区域 1, 1, 2, 3, 2, 4, 4。七次行动获得的保护价值依次为 4, 1, 3, 1, 1, 2, 1，总和为 13。

【样例 2】

见选手目录下的 *repair/repair2.in* 和 *repair/repair2.ans*。
该组样例符合测试点 1 ~ 5 的数据范围。

【样例 3】

见选手目录下的 *repair/repair3.in* 和 *repair/repair3.ans*。
该组样例符合测试点 6 ~ 10 的数据范围。

【样例 4】

见选手目录下的 *repair/repair4.in* 和 *repair/repair4.ans*。
该组样例符合测试点 11 ~ 15 的数据范围。

【样例 5】

见选手目录下的 *repair/repair5.in* 和 *repair/repair5.ans*。

该组样例符合测试点 16 ~ 20 的数据范围。

【数据范围】

对于 100% 的数据，保证 $1 \leq T \leq 100$ ， $1 \leq n, k \leq 2 \times 10^5$ ，单个测试点内 $\sum n \leq 2 \times 10^5$ ， $1 \leq a_i, b_i \leq 10^9$ 。

测试点编号	n, k	特殊性质
1 ~ 5	≤ 10	无
6 ~ 10	$\leq 2 \times 10^3$	无
11 ~ 15	$\leq 2 \times 10^5$	是
16 ~ 20	$\leq 2 \times 10^5$	无

特殊性质：保证 $b_1 \leq b_2 \leq \dots \leq b_n$ 。

风眼定位 (locate)

本题为交互题。

【题目背景】

沿岸设施恢复后，观测阵列重新接收到风暴内部的信号。tfbz 对照海图确认，最强信号所在的位置就是风眼，于是准备借助尚未完全修复的观测装置确定风眼的位置。

【题目描述】

观测阵列上共有 n 个位置，依次编号为 $1 \sim n$ 。第 i 个位置有一个隐藏的信号强度，所有位置的信号强度两两不同，其中信号最强的位置就是 tfbz 要寻找的风眼。

受损的观测装置无法直接找出一段区域内最强的信号。每次观测时，tfbz 可以选定两个整数 l, r ，其中 $1 \leq l < r \leq n$ 。观测装置会比较位置 $l, l+1, \dots, r$ 的信号，并告诉 tfbz 其中第二强的信号位于整个阵列中的哪个位置。

tfbz 可以进行若干次观测。每次观测结束后，tfbz 都可以根据已经得到的结果决定下一次观测的区域。最后，tfbz 需要指出整个阵列中信号最强的位置。

每次启动观测装置都需要消耗一枚 Gioush 大队珍藏的魔力晶石。为了给抵御风暴留下足够的材料，tfbz 希望尽可能减少观测次数。你需要帮助 tfbz 制定观测策略，确定风眼所在的位置。

【实现细节】

选手不需要，也不应实现 main 函数。

选手需要确保提交的程序包含头文件 `locate.h`，即在程序开头加入以下代码：

```
1 #include "locate.h"
```

选手需要在提交的程序源文件 `locate.cpp` 中实现以下两个函数：

```
1 void Init(int c, int T);
```

- c, T 分别表示测试点编号与测试数据组数。 $c = 0$ 表示该测试点为样例。
- 对于每个测试点，该函数会在程序开始运行时被交互库调用恰好一次。

```
1 int Locate(int c, int n);
```

- c, n 分别表示测试点编号与观测位置的数量。
- 该函数需要返回一个正整数，表示整个阵列中信号强度最大的位置。
- 对于每组测试数据，该函数会被交互库调用恰好一次。

选手可以通过调用以下函数进行一次询问：

```
1 int Query(int l, int r);
```

- l, r 表示 tfbz 本次观测区域的左右端点。选手需要确保 $1 \leq l < r \leq n$ 。
 - 该函数会返回位置 $l, l+1, \dots, r$ 中信号强度第二大的位置编号，具体含义如【题目描述】中所示。
 - 选手需要确保交互库每次调用 `Locate` 时，调用该函数的次数不超过 200。
- 交互库运行所需的时间与空间均计入本题的时间与空间限制。

本试题目录下的 `template_locate.cpp` 是提供的示例代码，选手可以参考并实现自己的程序。

【测试程序方式】

选手可以在本题目录下使用如下命令编译得到可执行文件：

```
1 g++ grader.cpp locate.cpp -o locate -O2 -std=c++14 -static
```

对于编译得到的可执行文件 `locate`：

- 可执行文件将从标准输入读入以下格式的数据：
 - 第一行包含两个非负整数 c, T ，分别表示测试点编号与测试数据的组数。
 $c = 0$ 表示该测试点为样例。
 - 接下来对于每组测试数据，第一行包含一个正整数 n 。
 - 第二行包含一个 $1 \sim n$ 的排列，排列中的第 i 个数表示位置 i 的信号强度。
- 可执行文件将输出以下格式的数据至标准输出：
 - 若 T 次调用 `Locate` 的返回值均正确，则：
 - * 输出的第一行为 **Correct!**。
 - * 输出的第二行为 **Max queries used: Q** ，其中 Q 表示所有测试数据中调用 `Query` 的次数的最大值。
 - * 输出的第三行为 **Score ratio: $P\%$** ，其中 P 表示按照【评分方式】计算出的得分比例。
 - 若至少一次调用 `Locate` 的返回值不正确，则只会输出一行 **wrong answer**。
- 若调用 `Query` 时传入的参数不符合要求，或调用次数超过上限，则可执行文件会向标准错误流输出错误信息并终止程序。

【样例 1 输入】

```
1 0 2
2 5
3 5 1 4 2 3
4 2
5 1 2
```

【样例 1 输出】

```
1 Correct!
2 Max queries used: 3
3 Score ratio: 100%
```

【说明/提示】**【样例 1 解释】**

对于第一组测试数据，依次调用 `Query(1,5)`、`Query(1,3)` 与 `Query(2,3)`，返回值分别为 3,3,2，于是可以确定最强信号位于位置 1。对于第二组测试数据，调用 `Query(1,2)` 得到 1，于是可以确定最强信号位于位置 2。两组测试数据中调用 `Query` 次数的最大值为 3。

【样例 2】

见选手目录下的 `locate/locate2.in` 和 `locate/locate2.ans`。
该组样例符合测试点 1 的数据范围。

【样例 3】

见选手目录下的 `locate/locate3.in` 和 `locate/locate3.ans`。
该组样例符合测试点 2 的数据范围。

【样例 4】

见选手目录下的 `locate/locate4.in` 和 `locate/locate4.ans`。
该组样例符合测试点 3 的数据范围。

【数据范围】

对于 100% 的数据，保证 $1 \leq T \leq 100$ ， $2 \leq n \leq 10^5$ ，所有信号强度两两不同。

测试点编号	分值	n	特殊性质
1	20	≤ 20	无
2	20	$\leq 10^5$	A
3	60	$\leq 10^5$	无

特殊性质 A：保证信号最强的位置为 1 或 n 。

【评分方式】

注意：

- 选手不应当通过非法方式获取交互库的内部信息，如试图直接读取各位置的信号强度，或直接与标准输入、输出流进行交互。此类行为将被视为作弊。
- 交互库不是适应性的，即每次调用 **Locate** 时所有位置的信号强度就已经确定，不会随交互过程变化。
- 最终的评测交互库与样例交互库的实现不同。

若 **Locate** 函数的返回值不正确，或调用 **Query** 时不符合以上要求，则相应测试点得 0 分。

在上述条件基础上：

- 对于每个测试点，设 Q 表示各组测试数据中调用 **Query** 次数的最大值， $score$ 表示该测试点的分值，则程序获得 $p(Q) \times score$ 分，其中 $p(Q)$ 按照下表计算。

询问次数 Q	$p(Q)$
$Q \leq 40$	100%
$41 \leq Q \leq 80$	70%
$81 \leq Q \leq 120$	40%
$121 \leq Q \leq 200$	20%

环锁解除 (circle)

【题目背景】

风眼的位置确定后, Ehundategh 驾船抵达风暴中心。紊乱的气流在海面上形成了一道环形空洞, 通向核心的入口正悬在空洞中央。

入口外侧的环锁仍在不停旋转。锁面上的每个符号都与一处封锁结构相连, 只有按照残存记录给出的顺序逐一解除, 入口才会停止排斥靠近的船只。

【题目描述】

环锁上有 n 个控制位, 围成一个首尾相接的圆环。每个控制位最初刻有一个 $1 \sim n$ 的符号, 并且每个符号恰好出现一次。

环锁始终指定其中一个位置为当前控制位。从当前控制位开始沿顺时针方向读到的符号依次构成排列 a 。另一份长度为 n 的排列 b 记录了符号的解除顺序, 其中 b_1 表示下一次必须解除的符号。

你可以进行任意次下列操作:

- 将排列 a 循环左移一位。若操作前 $a_1 \neq 0$, 需要花费 x 。
- 将排列 a 循环右移一位。若操作前 $a_1 \neq 0$, 需要花费 y 。
- 交换 x, y , 需要花费 z 。
- 若 $a_1 = b_1$, 将排列 b 循环左移一位, 同时令 $a_1 = 0$, 不花费代价。

循环左移会将 $a_1, a_2, \dots, a_n$ 变为 $a_2, a_3, \dots, a_n, a_1$ 。循环右移会将其变为 $a_n, a_1, a_2, \dots, a_{n-1}$ 。

旋转操作的代价由操作前的 a_1 决定。已经变为 0 的位置不会消失, 仍会与其他位置一起旋转。交换 x, y 后, 之后的旋转操作按照交换后的数值计算, 直到再次交换。

请你求出使排列 a 中所有元素均变为 0 的最小代价。

【输入格式】

从文件 `circle.in` 中读入数据。

本题包含多组测试数据。

输入的第一行包含两个非负整数 c, T , 分别表示测试点编号与测试数据的组数。 $c = 0$ 表示该测试点为样例。

对于每组测试数据:

第一行包含四个正整数 n, x, y, z 。

第二行包含 n 个整数, 表示排列 a 。

第三行包含 n 个整数, 表示排列 b 。

【输出格式】

输出到文件 `circle.out` 中。

对于每组测试数据输出一行一个整数，表示解除环锁所需的最小代价。

【样例 1 输入】

```
1 0 4
2 3 1 2 5
3 1 2 3
4 1 2 3
5 3 1 2 5
6 1 2 3
7 3 2 1
8 4 3 5 2
9 2 4 1 3
10 4 1 3 2
11 5 2 7 4
12 4 1 5 2 3
13 2 5 1 4 3
```

【样例 1 输出】

```
1 0
2 2
3 3
4 6
```

【说明/提示】**【样例 1 解释】**

第一组测试数据中，初始时可以免费解除符号 1。此后每次循环左移前，当前控制位都已经被解除，因此剩余符号也可以依次免费解除，答案为 0。

【样例 2】

见选手目录下的 *circle/circle2.in* 和 *circle/circle2.ans*。
该组样例符合测试点 1 ~ 5 的数据范围。

【样例 3】

见选手目录下的 *circle/circle3.in* 和 *circle/circle3.ans*。
该组样例符合测试点 6 ~ 10 的数据范围。

【样例 4】

见选手目录下的 *circle/circle4.in* 和 *circle/circle4.ans*。

该组样例符合测试点 11 ~ 19 的数据范围。

【样例 5】

见选手目录下的 *circle/circle5.in* 和 *circle/circle5.ans*。

该组样例符合测试点 20 ~ 25 的数据范围。

【数据范围】

对于 100% 的数据，保证 $1 \leq T \leq 20$ ， $1 \leq n \leq 10^6$ ，单个测试点内 $\sum n \leq 10^6$ ，排列 a, b 均由 $1 \sim n$ 恰好各出现一次构成， $1 \leq x, y, z \leq 10^6$ 。

测试点编号	n	特殊性质
1 ~ 5	≤ 10	无
6 ~ 10	$\leq 10^6$	是
11 ~ 19	$\leq 10^3$	无
20 ~ 25	$\leq 10^6$	无

特殊性质：保证 $x = y = z$ 。

风暴止息 (calm)

【题目背景】

环锁开启后, CodeDay 进入风暴核心。维持海域稳定的定风楔已经大面积失稳, 残留能量仍在不断改变它们的状态。

核心的控制阵式只能在随机模式与人工接管之间切换。过早接管会超出控制系统能够承受的范围, CodeDay 只能等待局面足够接近稳定, 再完成最后的修复。

【题目描述】

风暴核心共有 n 枚定风楔, 依次编号为 $1 \sim n$ 。每枚定风楔有稳定与失稳两种状态, 分别用 0 与 1 表示。目标是使所有定风楔均处于稳定状态。

启动编号为 i 的控制阵式, 会翻转所有编号为 i 的正约数的定风楔状态。也就是说, 稳定状态会变为失稳状态, 失稳状态会变为稳定状态。

例如, 启动编号为 6 的控制阵式时, 编号为 1, 2, 3, 6 的定风楔状态会同时翻转, 其他定风楔的状态保持不变。

同一个控制阵式可以被启动多次。每次启动后, 系统都会根据新的定风楔状态重新判断接下来的行动方式。

对于任意局面, 定义其最少操作数为从当前状态出发, 使所有定风楔稳定所需的最少控制阵式启动次数。

最少操作数只由当前局面决定, 与此前启动过哪些控制阵式无关。当所有定风楔已经稳定时, 最少操作数为 0。

CodeDay 按照以下方式行动:

- 如果当前局面的最少操作数不超过 k , CodeDay 会选择操作次数最少的方案完成剩余操作。
- 否则, 系统会在 $1 \sim n$ 中等概率随机选择一个编号, 并启动对应的控制阵式。

随机模式下的每次选择彼此独立, 同一个编号可能被连续选中。每次随机启动结束后, 系统都会重新计算当前局面的最少操作数。

一旦最少操作数不超过 k , CodeDay 便立即接管系统。此后不再进行随机选择, CodeDay 会直接按照操作次数最少的方案使全部定风楔稳定。

如果初始局面的最少操作数已经不超过 k , CodeDay 会从一开始就直接接管, 不会执行任何随机操作。如果随机操作使局面第一次满足接管条件, 则该次随机操作仍计入总操作次数。

所有可能的随机过程共同决定最终操作次数。请你求按照上述方式使所有定风楔稳定所需操作次数的期望。

可以证明, 这个期望乘以 $n!$ 后一定为整数。你只需要输出该整数对 998,244,353 取模的结果。

【输入格式】

从文件 *calm.in* 中读入数据。

本题包含多组测试数据。

输入的第一行包含两个非负整数 c, T ，分别表示测试点编号与测试数据的组数。
 $c = 0$ 表示该测试点为样例。

对于每组测试数据：

第一行包含两个整数 n, k 。

第二行包含 n 个整数，第 i 个整数表示编号为 i 的定风楔初始状态。

【输出格式】

输出到文件 *calm.out* 中。

对于每组测试数据输出一行一个整数，表示所求期望乘以 $n!$ 后对 998,244,353 取模的结果。

【样例 1 输入】

```
1 0 4
2 4 0
3 0 0 0 0
4 3 3
5 1 0 1
6 2 0
7 1 0
8 3 1
9 0 1 1
```

【样例 1 输出】

```
1 0
2 6
3 6
4 18
```

【说明/提示】

【样例 1 解释】

第一组测试数据中，所有定风楔已经稳定，因此答案为 0。

第二组测试数据的最少操作数为 1，不超过 k 。操作次数乘以 $3!$ 等于 6。

第三组测试数据中，完成操作所需次数的期望为 3。将其乘以 $2!$ 后得到 6。

【样例 2】

见选手目录下的 *calm/calms2.in* 和 *calm/calms2.ans*。

该组样例符合测试点 1 ~ 5 的数据范围。

【样例 3】

见选手目录下的 *calm/calms3.in* 和 *calm/calms3.ans*。

该组样例符合测试点 6 ~ 10 的数据范围。

【样例 4】

见选手目录下的 *calm/calms4.in* 和 *calm/calms4.ans*。

该组样例符合测试点 11 ~ 15 的数据范围。

【样例 5】

见选手目录下的 *calm/calms5.in* 和 *calm/calms5.ans*。

该组样例符合测试点 16 ~ 19 的数据范围。

【样例 6】

见选手目录下的 *calm/calms6.in* 和 *calm/calms6.ans*。

该组样例符合测试点 20 ~ 25 的数据范围。

【数据范围】

对于 100% 的数据，保证 $1 \leq T \leq 20$ ， $1 \leq n \leq 10^5$ ， $0 \leq k \leq n$ ，单个测试点内 $\sum n \leq 10^5$ ，每枚定风楔的状态均为 0 或 1。

测试点编号	n	特殊性质
1 ~ 5	≤ 20	无
6 ~ 10	≤ 300	无
11 ~ 15	$\leq 5 \times 10^3$	无
16 ~ 19	$\leq 10^5$	是
20 ~ 25	$\leq 10^5$	无

特殊性质：保证每组测试数据初始局面的最少操作数不超过 k 。